

Approximation Algorithm

Dr. M. Nalliah
Associate Professor
School of Computer Science and Engineering,
Vellore Institute of Technology
Vellore-632014. *nalliah.moviri@vit.ac.in*



April 10, 2026

Approximation Algorithms

- How do we deal with problems where we don't have an efficient algorithm?
- One option: heuristics
- But we'd like some guarantee: the answer we get should **never** be **too** far from the optimal.
- → Approximation Algorithms

The desired statement

Definitions:

- Let $A_P(I)$ be the value of the solution using algorithm A to instance I of some minimization problem P .
- Let $OPT_P(I)$ be the optimal (smallest) solution for instance I .

Goal: To say we have an approximation algorithm for a minimization problem P , we want to prove something like:

$$A_P(I) \leq \alpha(|I|) \cdot OPT_P(I)$$

for some function $\alpha(|I|)$.

$\alpha(n)$ might be a constant like “2” or maybe $O(\log n)$, etc.

Approximation Guarantee

For any instance I , $A(I) \leq \alpha(|I|) \cdot OPT(I)$.

Clearly, $\alpha \geq 1$ (for minimization problems) because we can't have a solution smaller than the optimal.

Want $\alpha(\cdot)$ to be as small as possible.

For example, if $\alpha(n) = 2$, we have the statement that the solution returned by our greedy algorithm is never more than twice as large as the optimal.

Analysis Problem: We don't know the optimal, so how do we compare against it?

Insight: A *lower bound* on the optimal works almost as well:

- Suppose we know that $B(I) \leq OPT(I)$ for some function B .
- If we can prove $A(I) \leq \alpha B(I)$, then that immediately implies that $A(I) \leq \alpha OPT(I)$.

Lower Bounds, Picture

- - - - αOPT

- - - - Bound αB

- - - - Our algorithm A

- - - - OPT

- - - - Lower Bound B



Approximation for NP-Complete Problems

Suppose we are given an NP-complete problem to solve. Even though (assuming $P = NP$) we can't hope for a polynomial-time algorithm that always gets the best solution, can we develop polynomial-time algorithms that always produce a “pretty good” solution?

Suppose we are given a problem for which (perhaps because it is NP-complete) we can't hope for a fast algorithm that always gets the best solution. Can we hope for a fast algorithm that guarantees to get at least a “pretty good” solution? E.g., can we guarantee to find a solution that's within 10% of optimal? If not that, then how about within a factor of 2 of optimal? Or, anything non-trivial? As seen in the last lectures, the class of NP-complete problems are all equivalent in the sense that a polynomial-time algorithm to solve any one of them would imply a polynomial-time algorithm to solve all of them (and, moreover, to solve any problem in NP). However, the difficulty of getting a good approximation to these problems varies quite a bit. In this lecture we will examine several important NP-complete problems and look at to what extent we can guarantee to get approximately optimal solutions, and by what algorithms.

Definition 21.1 Vertex-Cover

Given a graph G , find the smallest set of vertices such that every edge is incident to at least one of them.

Decision problem: “Given G and integer k , does G contain a vertex cover of size $\leq k$?”

For instance, this problem is like asking: what is the fewest number of guards we need to place in a museum in order to cover all the corridors. As we saw last time, this problem is NP-hard. However, it turns out that for any graph G we can at least get within a factor of 2.

Let's start first, though, with some strategies that don't work.

Example

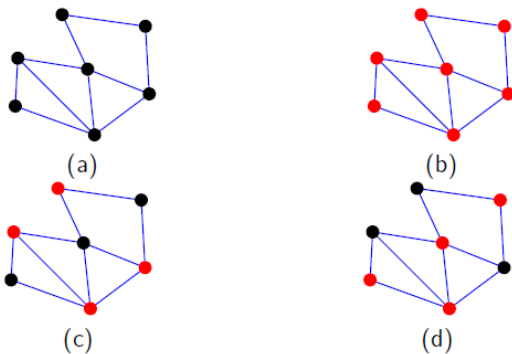


Figure: (a) An undirected graph (b) A trivial vertex cover (c) A vertex cover (d) An other vertex cover

Vertex-Cover Approximation Algorithms

Alg #1

Pick an arbitrary vertex with at least one uncovered edge incident to it, put it into the cover, and repeat.

What would be a bad example for this algorithm? [Answer: how about a star graph]

Alg #2

How about picking the vertex that covers the most uncovered edges. This is very natural, but unfortunately it turns out this doesn't work either, and it can produce a solution $\Omega(\log n)$ times larger than optimal.

How can we get factor of 2? It turns out there are actually several ways. We will discuss here two quite different algorithms. Interestingly, while we have several algorithms for achieving a factor of 2, nobody knows if it is possible to efficiently achieve a factor 1.99.

Vertex Cover: 2-Approximation

Definition

An algorithm for Vertex Cover is said to be a **factor-2 approximation** if the size of the cover it produces is at most twice the size of the optimal (minimum) vertex cover.

Formal Guarantee

Let OPT = size of the minimum vertex cover. Let ALG = size of the cover produced by the algorithm.

$$ALG \leq 2 \cdot OPT$$

Intuition

- The algorithm may not always find the smallest possible cover. - But it guarantees that the cover is never more than twice as large as the optimal.
- Example: If $OPT = 10$, then $ALG \leq 20$.

Algorithm 1

Pick an arbitrary edge. We know any vertex cover must have at least 1 endpoint of it, so let's take *both* endpoints. Then, throw out all edges covered and repeat. Keep going until there are no uncovered edges left.

Theorem

The above algorithm is a factor 2 approximation to Vertex-Cover.

Proof

What Algorithm 1 finds in the end is a matching (a set of edges no two of which share an endpoint) that is “maximal” (meaning that you can't add any more edges to it and keep it a matching). This means if we take both endpoints of those edges, we must have a vertex cover.

In particular, if the algorithm picked k edges, the vertex cover found has size $2k$. But, *any* vertex cover must have size at least k since it needs to have at least one endpoint of each of these edges, and since these edges don't touch, these are k different vertices.

So the algorithm is a 2-approximation as desired. ■

Vertex Cover: Algorithm 2 (LP Relaxation)

Here is now another 2-approximation algorithm for Vertex Cover:

Algorithm 2

First, solve a fractional version of the problem. Have a variable x_i for each vertex with constraint $0 \leq x_i \leq 1$. Think of $x_i = 1$ as picking the vertex, and $x_i = 0$ as not picking it, and in-between as “partially picking it”.

Then for each edge (i, j) , add the constraint that it should be covered in that we require $x_i + x_j \geq 1$. Then our goal is to minimize $\sum_i x_i$.

We can solve this using linear programming. This is called an “LP relaxation” because any true vertex cover is a feasible solution, but we’ve made the problem easier by allowing fractional solutions too.

So, the value of the optimal solution now will be at least as good as the smallest vertex cover, maybe even better (i.e., smaller), but it just might not be legal any more. [Give examples of triangle-graph and star-graph]

Now that we have a super-optimal fractional solution, we need to somehow convert that into a legal integral solution. We can do that here by just picking each vertex i such that $x_i \geq 1/2$.

This step is called **rounding of the linear program** (which literally is what we are doing here by rounding the fraction to the nearest integer — for other problems, the “rounding” step might not be so simple).

Vertex cover algorithm

APPROX-VERTEX-COVER(G)

```
1   $C = \emptyset$ 
2   $E' = G.E$ 
3  while  $E' \neq \emptyset$ 
4      let  $(u, v)$  be an arbitrary edge of  $E'$ 
5       $C = C \cup \{u, v\}$ 
6      remove from  $E'$  every edge incident on either  $u$  or  $v$ 
7  return  $C$ 
```


Approximation Algorithm for Vertex Cover

Theorem

The algorithm APPROX-VERTEX-COVER produces a vertex cover whose size is at most twice that of an optimal vertex cover.

Proof.

The set C returned by APPROX-VERTEX-COVER is a vertex cover, since the algorithm loops until every edge in $G.E$ is covered by some vertex in C .

Let A denote the set of edges chosen in line 4 of the algorithm. Any vertex cover, in particular an optimal cover C^* , must include at least one endpoint of each edge in A . No two edges in A share an endpoint, since once an edge is picked, all other edges incident on its endpoints are deleted from E' . Thus, no two edges in A are covered by the same vertex from C^* . Therefore,

$$|C^*| \geq |A| \quad (35.2)$$



Proof.

Each execution of line 4 picks an edge whose endpoints are not yet in C , so

$$|C| = 2|A| \tag{35.3}$$

Combining (35.2) and (35.3), we obtain

$$|C| = 2|A| \leq 2|C^*|,$$

which proves the theorem. □

Set-Covering Problem

Definition

An instance $(X, \mathcal{F})$ of the *set-covering problem* consists of:

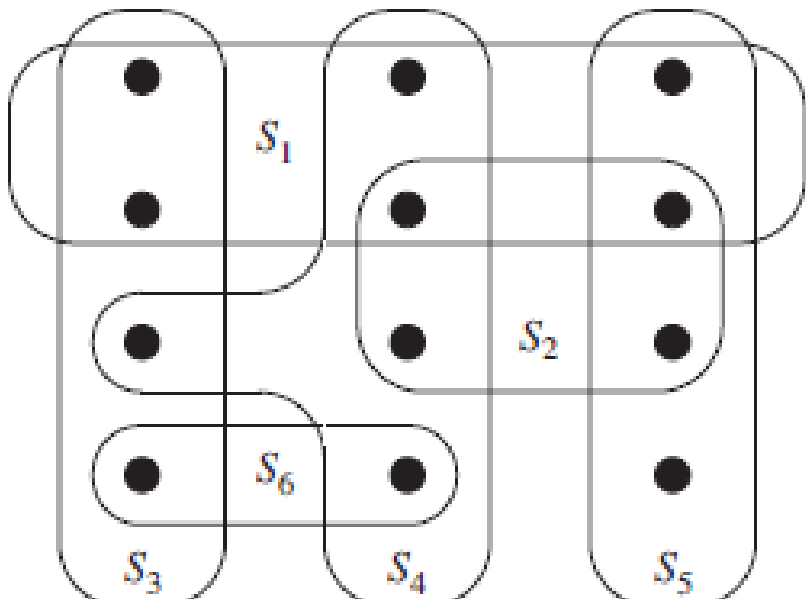
- A finite set X .
- A family $\mathcal{F}$ of subsets of X , such that every element of X belongs to at least one subset in $\mathcal{F}$:

$$X = \bigcup_{S \in \mathcal{F}} S.$$

We say that a subset $S \in \mathcal{F}$ *covers* its elements. The problem is to find a minimum-size subfamily $\mathcal{C} \subseteq \mathcal{F}$ whose members cover all of X :

$$X = \bigcup_{S \in \mathcal{C}} S.$$

Example



Example

Consider an instance $(X, \mathcal{F})$ of the set-covering problem, where:

- X consists of 12 black points.
- $\mathcal{F} = \{S_1, S_2, S_3, S_4, S_5, S_6\}$.

A minimum-size set cover is:

$$\mathcal{C} = \{S_3, S_4, S_5\}, \quad |\mathcal{C}| = 3.$$

The greedy algorithm produces a cover of size 4, selecting either:

$$\{S_1, S_4, S_5, S_3\} \quad \text{or} \quad \{S_1, S_4, S_5, S_6\}.$$

Definition

Set Cover Problem: Given a set system $\Sigma = (X, S)$, where $S = \{s_1, \dots, s_m\}$, compute a set $C \subseteq \{1, \dots, m\}$ of minimum cardinality such that

$$X = \bigcup_{i \in C} s_i$$

Example

Universe:

$$U = \{1, 2, 3, 4, 5\}$$

Subsets:

$$S_1 = \{1, 2\}, \quad S_2 = \{2, 3, 4\}, \quad S_3 = \{4, 5\}, \quad S_4 = \{1, 3, 5\}$$

Find the smallest number of subsets whose union covers U .

① $U = \{1, 2, 3, 4, 5\}$ must be fully covered.

② Possible covers:

$$S_1 \cup S_2 \cup S_3 = \{1, 2, 3, 4, 5\} \quad (3 \text{ sets})$$

$$S_2 \cup S_4 = \{1, 2, 3, 4, 5\} \quad (2 \text{ sets})$$

③ Minimum cover uses 2 sets.

Greedy Approximation Algorithm for Set Cover

A greedy approximation algorithm

The greedy method works by picking, at each stage, the set S that covers the greatest number of remaining elements that are uncovered.

GREEDY-SET-COVER($X, \mathcal{F}$)

```
1   $U = X$ 
2   $\mathcal{C} = \emptyset$ 
3  while  $U \neq \emptyset$ 
4      select an  $S \in \mathcal{F}$  that maximizes  $|S \cap U|$ 
5       $U = U - S$ 
6       $\mathcal{C} = \mathcal{C} \cup \{S\}$ 
7  return  $\mathcal{C}$ 
```

In

the above example, The greedy algorithm produces a cover of size 4, selecting either:

$$\{S_1, S_4, S_5, S_3\} \quad \text{or} \quad \{S_1, S_4, S_5, S_6\}.$$

Greedy Set Cover Algorithm

- Let U = set of remaining uncovered elements.
- Let $\mathcal{C}$ = cover being constructed.
- **Step 4:** Choose subset $S \in \mathcal{F}$ that covers the largest number of uncovered elements (ties broken arbitrarily).
- **Step 5:** Remove elements of S from U .
- **Step 6:** Add S to $\mathcal{C}$.
- Repeat until $U = \emptyset$.
- When the algorithm terminates:
 - $\mathcal{C}$ contains a subfamily of $\mathcal{F}$.
 - This subfamily covers the entire set X .

Implementation and Complexity of Greedy-Set-Cover

Implementation

The algorithm GREEDY-SET-COVER can be implemented to run in time polynomial in the sizes of the sets:

$$|X| \quad \text{and} \quad |\mathcal{F}|.$$

Complexity Analysis

- The number of iterations of the loop (lines 3–6) is bounded above by:

$$\min(|X|, |\mathcal{F}|).$$

- The loop body can be implemented to run in time:

$$O(|X||\mathcal{F}|).$$

- Therefore, a simple implementation runs in time:
 $O(|X||\mathcal{F}| \min(|X|, |\mathcal{F}|)).$

Analysis of Greedy-Set-Cover

Performance Guarantee

We now show that the greedy algorithm returns a set cover that is not too much larger than an optimal set cover.

Harmonic Numbers

For convenience, we denote the d th harmonic number by:

$$H(d) = \sum_{i=1}^d \frac{1}{i}.$$

As a boundary condition:

$$H(0) = 0.$$

- $H(k)$ denotes the k -th harmonic number:

$$H(k) = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{k}.$$

- The approximation ratio depends on the maximum set size in $\mathcal{F}$.
- Guarantees near-optimal coverage in polynomial time.

Theorem 35.4

Theorem (Greedy-Set-Cover)

GREEDY-SET-COVER is a polynomial-time $\rho(n)$ -approximation algorithm, where

$$\rho(n) = H(\max\{|S| : S \in \mathcal{F}\}).$$

- GREEDY-SET-COVER runs in polynomial time.
- At each step, it selects the set covering the largest number of uncovered elements.
- The process terminates when all elements are covered.
- Assign cost 1 to each set chosen by the algorithm.
- Distribute this cost evenly among elements covered for the first time.
- Each element x receives a cost c_x only once.

$$c_x = \frac{1}{|S_i - (S_1 \cup \dots \cup S_{i-1})|}$$

- Total cost equals size of greedy cover:

$$|\mathcal{E}| = \sum_{x \in X} c_x \quad (35.9)$$

- Each element lies in some set of the optimal cover $\mathcal{E}^*$:

$$\sum_{S \in \mathcal{E}^*} \sum_{x \in S} c_x = \sum_{x \in X} c_x \quad (35.10)$$

- Combining:

$$|\mathcal{E}| \leq \sum_{S \in \mathcal{E}^*} \sum_{x \in S} c_x \quad (35.11)$$

- For any set $S \in \mathcal{F}$:

$$\sum_{x \in S} c_x \leq H(|S|) \quad (35.12)$$

- $H(m)$ is the m -th harmonic number:

$$H(m) = 1 + \frac{1}{2} + \cdots + \frac{1}{m}$$

- Define $u_i =$ uncovered elements of S after i steps.
- Show:

$$\sum_{x \in S} c_x \leq \sum_{i=1}^k (u_{i-1} - u_i) \cdot \frac{1}{u_{i-1}}$$

- Bound further using telescoping sums.

$$\begin{aligned}\sum_{x \in S} c_x &= \sum_{i=1}^k (H(u_{i-1}) - H(u_i)) = H(u_0) - H(u_k) \\ &= H(|S|) \quad \text{since } u_0 = |S|, u_k = 0\end{aligned}$$

- Greedy-Set-Cover runs in polynomial time.
- Approximation ratio:

$$\rho(n) = H(\max\{|S| : S \in \mathcal{F}\})$$

Corollary 35.5

Corollary

GREEDY-SET-COVER is a polynomial-time $(\ln |X| + 1)$ -approximation algorithm.

Proof.

Use inequality (A.14) and Theorem 35.4. □

Inequality (A.14)

$$\sum_{k=1}^n \frac{1}{k} \leq \ln n + 1$$

Notes

- This inequality provides an upper bound for the harmonic series.
- Useful in analyzing approximation ratios of algorithms (e.g., Greedy-Set-Cover).
- Shows that growth of $\sum_{k=1}^n \frac{1}{k}$ is logarithmic in n .

- If $\max\{|S| : S \in \mathcal{F}\}$ is a small constant, the solution is at most a small constant factor larger than optimal.
- Example: Approximate vertex cover in graphs with maximum degree 3.
- In this case:

$$H(3) = 1 + \frac{1}{2} + \frac{1}{3} = \frac{11}{6}.$$

- Performance guarantee is slightly better than that of APPROX-VERTEX-COVER.

The traveling-salesman problem

Definition

- Input: Complete undirected graph $G = (V, E)$.
- Each edge $(u, v) \in E$ has cost $c(u, v) \geq 0$.
- Find a Hamiltonian cycle (tour) of G with minimum cost.

Notation

- For subset $A \subseteq E$:

$$c(A) = \sum_{(u,v) \in A} c(u, v).$$

- $c(A)$ = total cost of edges in A .

Triangle Inequality

- Cost function c satisfies:
 $c(u, w) \leq c(u, v) + c(v, w), \quad \forall u, v, w \in V$
- Cutting out intermediate stops never increases cost.
- Holds naturally for Euclidean distance and many other metrics.

Implications for TSP

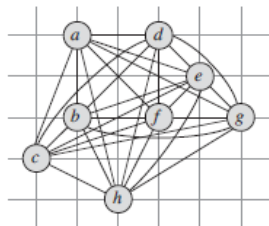
- Even with triangle inequality, TSP remains NP-complete.
- Exact polynomial-time solution is unlikely.
- Focus shifts to approximation algorithms.
- Example: 2-approximation algorithm exists under triangle inequality.

The traveling-salesman problem with the triangle inequality

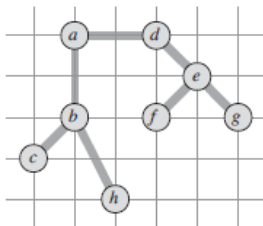
APPROX-TSP-TOUR(G, c)

- 1 select a vertex $r \in G.V$ to be a “root” vertex
- 2 compute a minimum spanning tree T for G from root r
using MST-PRIM(G, c, r)
- 3 let H be a list of vertices, ordered according to when they are first visited
in a preorder tree walk of T
- 4 return the hamiltonian cycle H

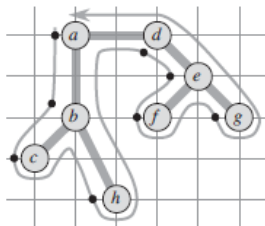
Example



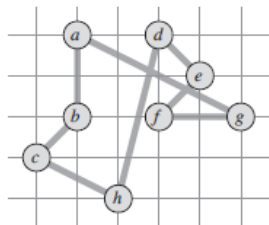
(a)



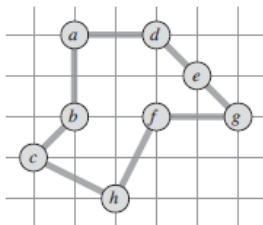
(b)



(c)



(d)



(e)

Steps of the Algorithm

- a Start with a complete undirected graph G .
- b Compute a minimum spanning tree T (e.g., using Prim's algorithm).
- c Perform a preorder walk of T . (c) A walk of T , starting at a . A full walk of the tree visits the vertices in the order $a, b, c, b, h, b, a, d, e, f, e, g, e, d, a$. A preorder walk of T lists a vertex just when it is first encountered, as indicated by the dot next to each vertex, yielding the ordering a, b, c, h, d, e, f, g .
- d Visit vertices in preorder order to form tour H . A tour obtained by visiting the vertices in the order given by the preorder walk, which is the tour H returned by APPROX-TSP-TOUR. Its total cost is approximately 19.074.
- e An optimal tour H^* for the original complete graph. Its total cost is approximately 14.715

Theorem (APPROX-TSP-TOUR)

APPROX-TSP-TOUR is a polynomial-time 2-approximation algorithm for the traveling-salesman problem with the triangle inequality.

Polynomial-Time Guarantee

- APPROX-TSP-TOUR runs in polynomial time.
- Based on MST construction and preorder traversal.
- Efficiently produces a Hamiltonian cycle.

Optimal Tour and MST

- Let H^* = optimal tour.
- Removing one edge from H^* yields a spanning tree.
- Thus:

$$c(T) \leq c(H^*) \quad (35.4)$$

where T is the MST.

Full Walk of MST

- Define $W =$ full walk of T .
- Each edge of T is traversed twice.
- Therefore:

$$c(W) = 2c(T) \quad (35.5)$$

Bounding the Walk

- From (35.4) and (35.5):

$$c(W) \leq 2c(H^*) \quad (35.6)$$

- The walk cost is within factor 2 of optimal.

Issue: Walk vs Tour

- W is not a tour — vertices may repeat.
- Triangle inequality allows shortcutting repeated visits:

$$c(u, w) \leq c(u, v) + c(v, w)$$

- Delete repeated vertices without increasing cost.

Preorder Walk

- Removing repeats yields preorder walk of T .
- This defines a Hamiltonian cycle H .
- Every vertex is visited exactly once.

Final Inequality

- Since H is obtained by deleting vertices from W :

$$c(H) \leq c(W) \quad (35.7)$$

- Combining (35.6) and (35.7):

$$c(H) \leq 2c(H^*)$$

- Completes the proof.

The general traveling-salesman problem

Theorem

If $P \neq NP$, then for any constant $\rho \geq 1$, there is no polynomial-time approximation algorithm with approximation ratio ρ for the general traveling-salesman problem.

Proof Strategy

- Proof by contradiction.
- Assume there exists a polynomial-time approximation algorithm A with ratio ρ .
- Show how A can solve the Hamiltonian-cycle problem in polynomial time.
- Contradicts NP-completeness unless $P = NP$.

Reduction Setup

- Input: Graph $G = (V, E)$ for Hamiltonian-cycle problem.
- Construct complete graph $G' = (V, E')$.
- Define edge costs:

$$c(u, v) = \begin{cases} 1 & \text{if } (u, v) \in E, \\ \rho|V| + 1 & \text{otherwise.} \end{cases}$$

Case 1: G has Hamiltonian Cycle

- Suppose H is a Hamiltonian cycle in G .
- Then in (G', c) , all edges of H have cost 1.
- Total cost of H :

$$c(H) = |V|.$$

Case 2: G has No Hamiltonian Cycle

- Any tour in G' must use at least one edge not in E .
- Such an edge costs $\rho|V| + 1$.
- Therefore, any tour has cost at least:

$$(\rho|V| + 1) + (|V| - 1) = \rho|V| + |V| > \rho|V|.$$

Gap Argument

- Tours corresponding to Hamiltonian cycles cost $|V|$.
- Tours without Hamiltonian cycles cost $> \rho|V|$.
- Gap factor: at least $\rho + 1$.
- Approximation algorithm A must distinguish these cases.

Finally

- If G has a Hamiltonian cycle, A returns it (cost $\leq \rho|V|$).
- If G has no Hamiltonian cycle, A returns a tour with cost $> \rho|V|$.
- Thus, A solves Hamiltonian-cycle problem in polynomial time.
- Contradiction: Hamiltonian-cycle is NP-complete.
- Therefore, no constant-factor approximation exists unless $P = NP$.